Experiment: - 2

Aim: To containerize application using Docker

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory:

What is Docker?

Docker is an open-source platform used to develop, package, ship, and run applications in containers. It allows developers to bundle an application along with all its dependencies, libraries, and configuration files into a single package, ensuring that the application runs consistently across different environments. Docker simplifies application deployment by eliminating compatibility issues between development, testing, and production systems.

What is Containerization?

Containerization is a virtualization technique in which an application and all its required dependencies are packaged into an isolated unit called a container. Containers share the host operating system's kernel but run independently from one another, making them lightweight and efficient compared to traditional virtual machines. Containerization ensures that applications behave the same regardless of the underlying operating system or infrastructure.

Difference between Image and Container?

Docker Image	Docker Container
A blueprint or template used to create containers.	A running instance of a Docker image.
It is read-only and cannot be changed directly.	It is read-write and can be modified while running.
Used to package the application, libraries, dependencies, and configuration.	Used to execute the packaged application.

Stored on the local system or downloaded from Docker Hub.	Created from an image and exists only after docker run.
Multiple containers can be created from the same image.	Each container is an independent instance of the image.
Created using docker build or downloaded using docker pull.	Created using docker run.
Images do not execute code by themselves.	Containers execute the application or process.
Example: ubuntu:latest, nginx, mysql.	Example: ubuntu_lab, web_server, db_container.

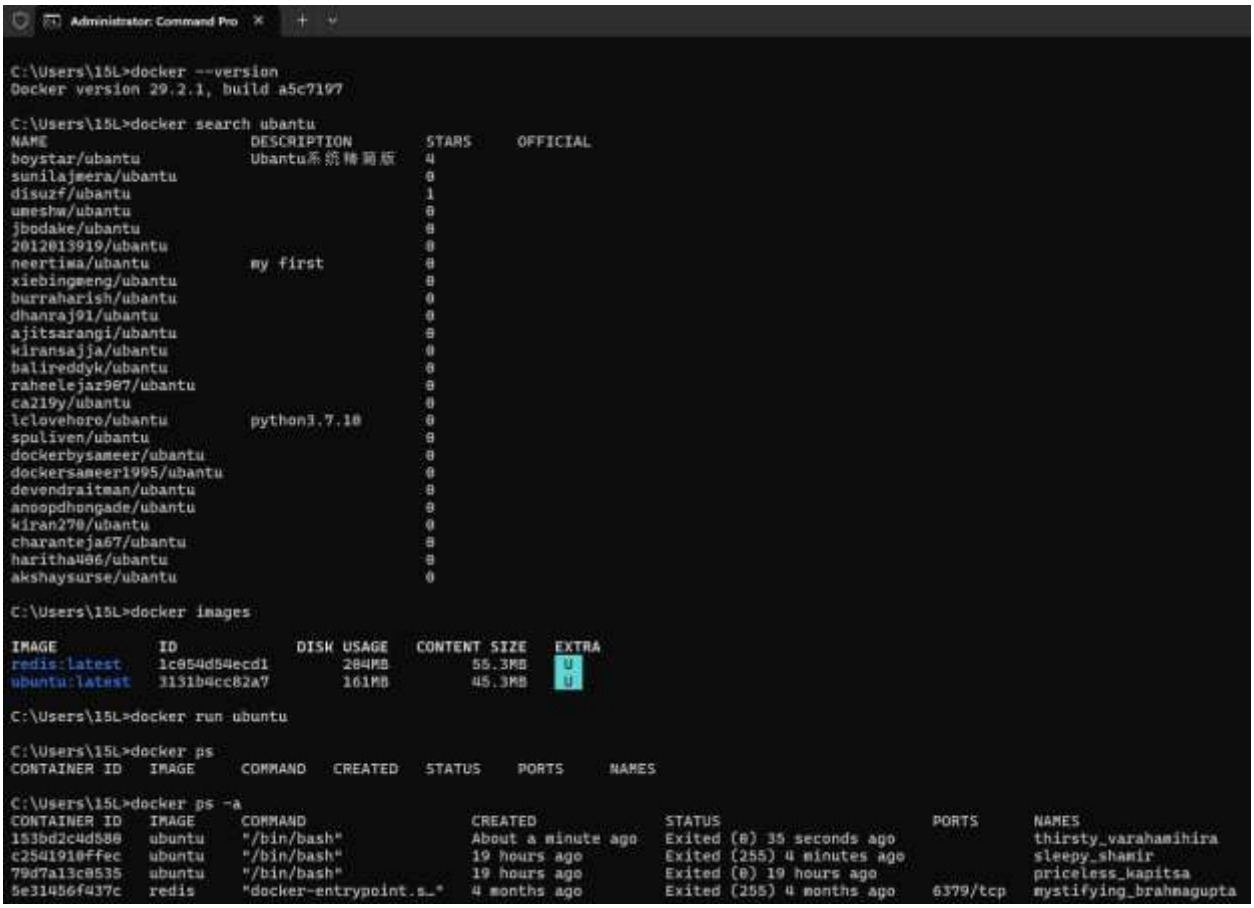
Commands:

Command	Description
docker --version	Displays the installed Docker CLI and Engine version.
docker info	Displays system-wide information about the Docker installation.
docker system df	Shows Docker disk usage by images, containers, volumes, and build cache.
docker search <term>	Searches Docker Hub for public images matching a keyword (e.g., docker search ubuntu).
docker pull <image>	Downloads an image from Docker Hub to your local system.
docker images or docker image ls	Lists all locally stored Docker images.

docker rmi <image_id/name>	Removes one or more Docker images from local storage.
docker build -t <tag> .	Builds a Docker image from a Dockerfile in the current directory.
docker run <image>	Creates and starts a new container from an image.
docker start <container>	Starts one or more stopped containers.
docker stop <container>	Gracefully stops a running container.
docker restart <container>	Restarts a container.
docker kill <container>	Forcefully stops a running container immediately.
docker rename <old_name> <new_name>	Renames an existing container.
docker ps	Lists all currently running containers.
docker ps -a	Lists all containers, including stopped and exited ones.
docker logs <container>	Displays the output logs of a container.
docker inspect <container>	Shows detailed JSON information about a container or image.
docker exec -it <container> <command>	Executes a command interactively inside a running container (e.g., docker exec -it ubuntu_lab bash).
docker cp <source> <destination>	Copies files or folders between the host and a container.
docker commit <container> <image_name>	Creates a new image from a modified container.
docker export <container> -o <filename.tar>	Exports a container's filesystem as a TAR archive.

docker import <filename.tar> <image_name>	Creates a Docker image from an exported TAR archive.
docker rm <container>	Removes a stopped container.
docker rm -f <container>	Forcefully removes a running container.
docker container prune	Removes all stopped containers.
docker network prune	Removes all unused Docker networks.
docker system prune	Removes unused containers, networks, dangling images, and build cache to free up space.

Output:




```
C:\Users\15L>docker search ubantu --filter stars=1000
NAME      DESCRIPTION   STARS     OFFICIAL

C:\Users\15L>docker exec -it myubuntu bash
root@175578fe3096:/# pwd
ls
whoami
cat /etc/os-release
/
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
root@175578fe3096:/# exit
exit
```

```
C:\Users\15L>docker start myubuntu
myubuntu

C:\Users\15L>docker stop myubuntu
myubuntu

C:\Users\15L>docker restart myubuntu
myubuntu

C:\Users\15L>docker kill myubuntu
myubuntu

C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\15L>docker start myubuntu
myubuntu

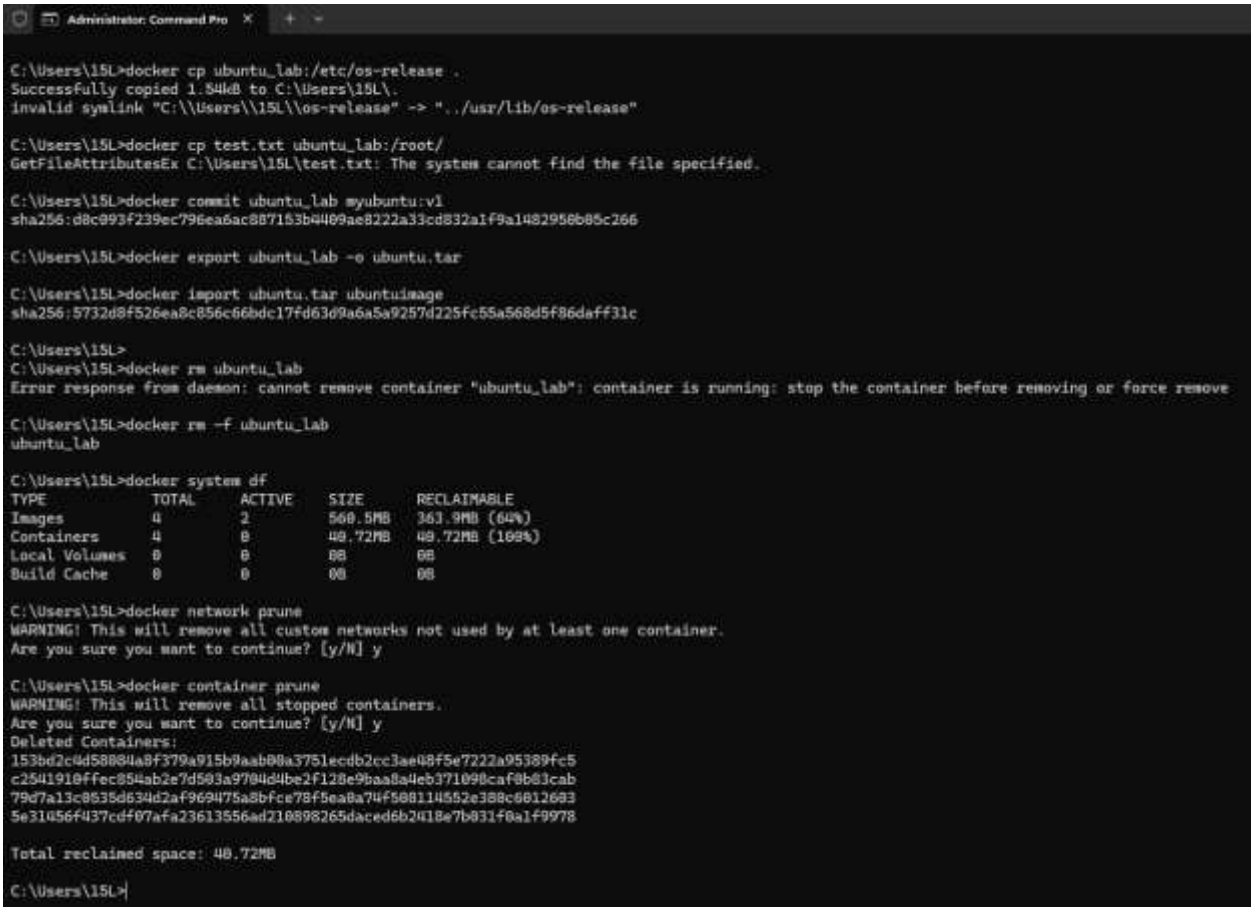
C:\Users\15L>docker rename myubuntu ubuntu_lab

C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS          PORTS     NAMES
175578fe3096   ubuntu    "/bin/bash"   8 minutes ago   Up 23 seconds             ubuntu_lab

C:\Users\15L>docker exec -it ubuntu_lab bash
root@175578fe3096:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@175578fe3096:/# exit
exit

C:\Users\15L>docker exec ubuntu_lab pwd
/
```





Conclusion:

Through this Docker experiment, I learned the fundamental concepts of containerization and gained practical experience with Docker commands. I understood the difference between Docker images and containers and learned how to manage images, create and run containers, inspect their status, execute commands inside containers, and perform cleanup operations. This experiment enhanced my understanding of how Docker simplifies application deployment by providing a consistent and isolated execution environment across different systems.